

Attention is Content-Addressable Memory: A Transformer Learns Pointer Dereferencing

The architecture-axis capstone of the inductive-bias series. The previous posts each committed to a structural prior about *which positions matter*: the CNN’s spatial weight sharing, the RNN’s recurrent state, the fixed-context LM’s bounded window. The transformer commits to something stranger and more flexible. It commits to a *mechanism* for deciding which positions matter, from data, position by position, and on the fly.

The architectural claim of this post is one sentence:

An attention head is a **learned content-addressable lookup**: queries match against keys to retrieve values, exactly as a pointer dereference reads memory by address. Stacking attention layers composes lookups into multi-hop reads. That is the entire structural prior of a transformer.

To make the claim concrete and falsifiable, we design a synthetic data-generating process whose structure *requires* content-addressable memory to model efficiently, then train small transformers on it and watch attention learn to do exactly what its mechanism is built for. The DGP has a single-lookup variant and a multi-hop variant, which lets us also demonstrate the second half of the claim: depth in a transformer corresponds to the number of composed lookups it can resolve.

This post does *two* things. The first half (sections 1 through 6) sets up the task, derives the architectural depth argument, and verifies it empirically at small scale. The second half (sections 7 through 10) is the part the post does not start out planning to do: when we try to *scale* the task by increasing the memory size, the transformer suddenly underperforms a brute-force MLP, and we have to debug *why*. The debugging turns out to teach the most useful lesson in the post.

This is a different worked example from the archived `docs/series/archive/transformer-text.md`, which trains a decoder-only character transformer on famous-authors text. The text post showcases the same architecture on a richer task; this one strips the task to its essentials so the inductive bias is visible in isolation. The architecture pieces (embedding, sinusoidal positional encoding, scaled dot-product attention, multi-head attention with causal mask, layer norm, residuals, GELU feedforward, output projection) are imported from `examples/transformer.py`, the same NumPy implementation the text post derived by hand. Only the training task differs.

1. Attention as a lookup

Scaled dot-product attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

does three things in sequence: compute pairwise dot products between queries and keys, softmax them into a distribution over past positions, then return the attention-weighted average of values. Read operationally, this is exactly what a content-addressable memory does:

- The query at position t encodes “what I am looking for.”
- The keys at past positions i encode “what I am.”
- The values at past positions encode “what I would return if you pick me.”

- The softmax picks a (soft) past position whose key best matches the query; the output is the corresponding value.

When the query is the encoding of a position (“I want position k back”), the keys encode positions, and the softmax becomes a (soft) hard-attention spike on the addressed past position. The value at that position is then returned. That is a pointer dereference.

This is structurally what a transformer wants to do. Whether it *succeeds* depends on whether the relevant address information is actually present in the query, which is where the depth story comes in.

2. A pointer-dereferencing task

The task is fixed-format and deliberately small, so the structure is visible. Each example is a 12-bit sequence:

[m_0 m_1 ... m_7]	[a_0 a_1 a_2]	[y]
8 memory bits	3 address	target

The 8 memory bits are random. The 3 address bits are a binary encoding of an index $a \in \{0, 1, \dots, 7\}$, most significant bit first. The target is $y = m_a$: the memory bit at the addressed position.

There are $2^{11} = 2048$ possible inputs. We generate 20,000 random examples for training (with replacement, so duplicates abound) and 2000 for held-out evaluation. The model sees the first 11 bits and predicts the 12th. The generator is in `examples/simple_pointer_dgp.py`.

This is the smallest task that *requires* content-addressable memory. The model must learn three steps:

1. Recognize that positions 8, 9, 10 are address bits.
2. Decode the 3 bits to an integer a .
3. Use a to attend to memory position m_a and emit that bit.

Step 3 is the dereference. It is dynamic in the sense that *which* memory position the model needs to read depends on the *content* of the address bits, which only become known at inference time.

3. Why one attention layer is not enough

A subtle but load-bearing point: a 1-layer, 1-head transformer *cannot* solve this task, for a structural reason.

The prediction at position 11 uses the output of attention at position 10 (the last input position). The query at position 10, before attention, depends only on the input at position 10: the single bit a_2 , plus a positional encoding. It does *not* depend on a_0 or a_1 , which are at positions 8 and 9.

The attention layer then computes a weighted sum of past values V_i , with weights set by $\text{softmax}(Q_{10} \cdot K_i)$. The weight at past memory position i depends on Q_{10} and K_i ; since Q_{10} is fixed given the inputs at position 10, the weights are *not* a function of the full address. The attention output is a weighted sum of memory values, but the weights cannot dynamically select position a .

The model can learn fixed attention patterns (always attend to memory position 3, say) but it cannot route to a position whose identity depends on bits 8 and 9. One layer is not enough to do dynamic addressing.

Two layers are. The first layer lets position 10’s hidden state incorporate information from positions 8 and 9 via attention. The second layer’s query at position 10 is then computed from a hidden state that contains all three address bits, so it can dynamically address memory positions. This is the precise reason transformer depth equals the number of composed lookups the model can do: each additional layer adds one more “stage” of dynamic routing.

4. Experiment 1: depth on a single lookup

We train three models on the single-lookup task, all with parameter counts in the same ballpark, all with Adam at learning rate 10^{-3} for 2000 iterations of batch size 32. Code is in `examples/pointer_experiments.py`.

Model	Layers	Heads	Parameters	Test accuracy
MLP baseline	(n/a)	(n/a)	3,018	1.000
Transformer	1	1	8,738	0.654
Transformer	2	1	17,282	1.000

Two observations.

The 1-layer transformer is stuck at ~65% accuracy, which is modestly above the 50% random baseline but nowhere near the task’s ceiling. This matches the structural argument from section 3 exactly: the model can learn fixed attention patterns and pick up some correlations (the address bit at position 10 already gives 1 bit of information about the answer), but it cannot do the dynamic lookup. Adding more parameters within 1 layer would not help, because the limitation is architectural.

The 2-layer transformer hits 100% accuracy with a clear phase transition. Training loss is at random (~ 0.69) for the first ~ 600 iterations, then drops sharply through iteration 1200, then sits at machine zero from iteration 1500 onward. This is the “grokking” shape that recurs whenever a model finds the right algorithm: a long plateau while it searches, then a fast collapse to perfect performance once it has it.

The MLP also hits 100%, but the why is more interesting than “memorization.” The MLP has hidden = 128 units, and at $M = 8$ with $2^A = 8$ address combinations that is roughly $128/8 = 16$ hidden units per address. With that capacity, the MLP can learn a brute-force *expert-per-address* decomposition: a few hidden units that fire exactly when (address bits = k AND memory bit $k = 1$), with the output layer summing the activations of the experts. This is structurally an algorithm, not memorization, but it scales linearly in the number of distinct addresses, so an MLP with hidden = 128 starts to break down once $2^A > 64$ or so. The transformer is *supposed* to scale linearly in M regardless of A , because its attention is content-addressable rather than expert-decomposed. Section 7 takes the obvious next step and tests whether this scaling story actually holds.

The training-curve shapes (MLP improves smoothly; transformer plateaus then groks) are the visual signature of the difference: brute-force algorithm fitting is a smooth interpolation, learning an attention algorithm is a discrete jump.

5. Multi-head attention and parallelism

The single-head experiments already work. What does adding more heads buy?

Conceptually, multi-head attention lets one layer run multiple attention operations in parallel and combine them. Each head has its own W_Q, W_K, W_V projections operating on a slice of d_{model} , so the heads can specialize: one head might attend to syntactic features, another to long-range dependencies, another to specific tokens. The output projection W_O combines them.

For our memory-pointer task at $M = 8$, the lookup is conceptually a single operation: there is one address to decode and one memory position to read. A second head does not add a qualitatively new capability at this scale. We verified empirically that a 2-layer 2-head model converges to the same 100% accuracy as a 2-layer 1-head model. The heads do not specialize because there is nothing to specialize on.

This story changes at larger M , where multi-head turns out to help for a reason we did not initially expect (related to the positional-encoding issue uncovered in section 9). In production transformers on language, head specialization is well-documented (induction heads, copy heads, syntactic heads). The synthetic task at $M = 8$ is just not large enough to force it.

6. Experiment 2: depth on a multi-hop lookup

For the depth-scaling story we use a pointer-to-pointer task: the memory bit at the addressed position, combined with some address bits, forms a *new* address used for a second lookup. Each example is 12 bits with the same layout as before, but the target is now

$$y = m_{f(m_a, a)}$$

where f combines m_a with the lower bits of a to form a new address. The exact combination is in `make_variant3` in `examples/simple_pointer_dgp.py`; the essential point is that the target depends on *two* lookups composed in sequence. To predict the target, the model must:

1. Decode the address bits.
2. Look up m_a (first lookup).
3. Combine m_a with the original address bits to form a new address.
4. Look up memory at that new address (second lookup).

A 2-layer transformer can in principle resolve the first lookup in layer 1 and the second in layer 2. A 3-layer transformer has one extra layer to spare. We train both on 20,000 examples, 2500 Adam iterations:

Model	Layers	Parameters	Test accuracy
Transformer	2	17,282	0.977
Transformer	3	25,826	0.995

The 2-layer model gets close to perfect (97.7%), and the 3-layer model essentially solves it (99.5%). Both grok within 2500 iterations. The marginal benefit of the third layer is small here because the task only needs two hops; a 3-hop task would push the gap between 2 and 3 layers wider. The trend, even on this small demonstration, is what the depth-equals-composed-lookups story predicts: more layers buy more hops.

7. Scaling beyond M=8, the puzzle

So far our story is: 1-layer fails by architectural argument, 2-layer succeeds in a clean grokking shape, multi-hop benefits from extra depth. The architectural prior of content-addressable memory seems to be doing exactly what theory predicts.

The natural next experiment: scale the memory size M up and watch the transformer beat the MLP cleanly. Attention’s parameter count is $O(M \cdot d^2)$ while the MLP’s brute-force expert-per-address scheme is $O(2^A \cdot \text{hidden})$, so as M grows (and with it $A = \lceil \log_2 M \rceil$), the MLP’s hidden-units-per-address budget shrinks while the transformer’s mechanism stays constant.

Empirically that is not what happens.

M	A	Input space	MLP (h=128)	Transformer (d=64, 2L, 6000 iters)
8	3	2^{11}	1.000	1.000
16	4	2^{20}	1.000	0.747
24	5	2^{29}	0.996	0.687
32	5	2^{37}	0.870	0.664
48	6	2^{54}	0.654	0.529
64	6	2^{70}	0.607	0.540

The MLP scales smoothly with M until it runs out of capacity at $M \geq 48$ (where $128/2^A = 2$ hidden units per address combination is too few). The transformer, expected to keep scaling, plateaus near chance at every $M \geq 16$. The MLP *wins* at $M = 16, 24, 32$.

Same picture under varying training data at fixed $M = 16$. The MLP shows the textbook sample-efficiency curve (more data, higher accuracy); the transformer plateaus near chance regardless of how much data it sees:

n_{train}	%-space	MLP (h=128)	Transformer (d=64, 2L)
200	0.019%	0.580	0.568
500	0.048%	0.662	0.599
1000	0.095%	0.733	0.573
2000	0.191%	0.864	0.627
5000	0.477%	0.9995	0.741
20000	1.907%	1.000	0.738

This was a puzzle. Either the theoretical claim about attention as content-addressable memory is *wrong* (a real possibility: maybe attention’s expressivity is not as cheap as the theory suggests), or something specific to our small-scale implementation is breaking the bias’s effectiveness. The MLP’s success at $M = 16$ on 5000 training examples covering 0.5% of the input space *does* require it to generalize, so this is not a case of “the task is too easy for memorization either.”

8. Investigation, four parallel hypotheses

To find what was actually wrong, we ran four independent investigations in parallel:

1. **Audit the code.** Look for a real bug in attention, layer norm, residuals, or the training loop.
2. **Sweep optimization recipes.** Try learning-rate warmup, gradient clipping, different optimizers, longer training.
3. **Try different supervision formats.** Maybe sparse supervision (loss only at the final position) is the bottleneck; standard language-model training has dense per-position supervision.
4. **Try architecture variants.** Different layer counts, head counts, d_{model} sizes, learned positional encoding, no causal mask.

Audit result: the math is correct. A numerical gradient check on the full pipeline in float64 (with $\varepsilon = 10^{-6}$) returns a worst relative error of 2.3×10^{-7} on parameters with nonzero true gradient. Attention forward/backward, LayerNorm, the multi-head split/merge, the FFN+GELU, residual connections, pre-norm structure, and the sparse-dlogits construction are all mathematically correct. The naive float32 check at $\varepsilon = 10^{-4}$ produces apparent errors of 10^{-1} , but those are float32 noise from catastrophic cancellation in the forward, not real disagreement.

Optimization result: no recipe rescues the model. LR warmup over 500 iters, gradient clipping, larger batches with matched compute, longer training (20000 iters), and lower or higher learning rates all stay within a few percentage points of the baseline at $M = 16$. The loss landscape’s plateau is not a tuning problem.

Supervision result: dense per-position supervision *hurts* (drops to 56.7%). Intermediate positions have random next-bit targets, and that noise drowns the one informative position. A 10x reweighted final position recovers most of the baseline (72.6%). Inserting a special [QUERY] token before the target gives the model an explicit prediction position and lifts to 81.6%, meaningfully better than the baseline, but still well short of the algorithm. Training longer (20000 iters) does not grok; the plateau is stable.

Architecture result: two variants reach the algorithmic ceiling.

Architecture variant	$M = 16$ test acc
Baseline (2L, 1H, sinusoidal PE)	0.747
More layers (3L, 1H)	0.733
More layers (4L, 1H)	0.729
Wider $d_{\text{model}} = 128$	0.751
No causal mask	0.746
Bigger FFN ($d_{\text{ff}} = 512$)	0.726
More heads ($n_{\text{heads}} = 4$)	0.983
Learned positional encoding	1.000

Depth, width, FFN size, and the causal mask are all non-issues. Two things matter: **the choice of positional encoding** and **how many attention heads are available**. Learned PE solves the task to 100% test accuracy; switching to 4 heads with sinusoidal PE gets to 98.3%. Both convergent findings point at the same underlying issue: at this scale, the model is fighting *positional* signal more than it is *content* signal.

9. The fix, positional-encoding scale

The audit pinpointed *why* the choice of positional encoding matters so much. The **Embedding** layer initializes its weight matrix at scale 0.02, which is the standard PyTorch default and is calibrated for large vocabularies. With a 2-token bit vocabulary, the input embedding scale stays at 0.02, while the sinusoidal positional encoding has entries of magnitude ~ 1 . The content signal is roughly $35\times$ smaller than the positional signal at the input.

Attention has to learn to discount the positional swamping before it can route based on content. At $M = 8$ this finishes in time; at $M \geq 16$ the optimization plateaus before the model has rescaled the embedding magnitudes to anything competitive with the PE.

There are two clean fixes:

1. **Rescale the embedding** so that content and position start at comparable magnitudes. The “Attention is All You Need” paper does this explicitly: it multiplies the embedding output by $\sqrt{d_{\text{model}}}$ before adding the positional encoding. A crude $25\times$ initial rescaling of the embedding weights lifts $M = 16$ from 67% to 79% with no other changes (verified during the audit).
2. **Use a learned positional encoding** instead of a sinusoidal one, initialized at the same small scale (0.02) as the embedding. Now content and position both start small, and the model can scale them together as training proceeds. This is what fixes the lookup task to 100% in our architecture sweep.

Both fixes target the same underlying issue: the content / position *scale balance* at the network’s input. Modern transformer libraries (PyTorch, JAX/Flax) use either learned PE or rotary embeddings, both of which sidestep the fixed-magnitude sinusoidal-versus-tiny-embedding problem. At production scale (large vocabularies, well-tuned init, warmup, etc.) the issue mostly disappears. At our small-experiment scale, it dominates.

The multi-head result (98.3% at $M = 16$ with 4 heads and sinusoidal PE) is consistent with this story too: with multiple heads, each head has its own projections and at least one head can specialize to a favorable Q/K geometry that compensates for the input-scale problem. With a single head, no such redundancy is available.

10. Verification, the inductive bias is partially confirmed

With the fix in hand we rerun the scaling experiment. **Same** architecture as baseline (2 layers, 1 head, $d_{\text{model}} = 64$, 6000 iters, $\text{lr} = 10^{-3}$, batch 32, 20000 training examples), only difference is the positional encoding (sinusoidal versus learned, both at the same 0.02 init scale):

M	Sinusoidal PE	Learned PE
16	0.747	1.000
24	0.687	0.605
32	0.664	0.645

At $M = 16$ the fix is dramatic: learned PE solves the lookup task to 100% test accuracy, exactly as the inductive-bias theory predicts.

At $M \geq 24$, however, learned PE alone does *not* generalize the fix. Both PE choices stay near chance at this scale. The scale-mismatch bottleneck that dominated at $M = 16$ is one of *several*

barriers; another (probably related to combining 5 or more address bits into a usable query vector at $d_{\text{model}} = 64$, with only one attention head) kicks in at larger M and is not addressed by the PE fix alone.

This nuance, frustrating to encounter but honest to report, *reinforces* the post’s core lesson rather than weakening it. The theoretical inductive bias of an architecture is **necessary but not sufficient**. Whether the architecture realizes its bias depends on a co-evolved set of choices that the theory glosses over: parameter initialization, positional encoding, optimizer state, learning-rate schedule, model width, layer count, training data volume, training duration. At production scale, established defaults handle most of these implicitly. At our small experimental scale, the right defaults have to be discovered one bottleneck at a time. We found one (positional encoding); the $M \geq 24$ result suggests there are more.

Production transformers handle the higher- M regime via much larger d_{model} (typically 512 to 2048), more attention heads (8 to 32), more layers (12 to 96), and warmup-plus-decay learning-rate schedules. We tested a scaled-down version of this **production recipe** on our task to see how far it closes the gap.

The recipe (in `examples/pointer_kitchen_sink.py`):

- $d_{\text{model}} = 128$ (vs. 64 in the baseline runs)
- 4 attention heads (vs. 1)
- 2 layers (unchanged)
- Learned PE at small init scale (matching the embedding)
- $\text{LR} = 10^{-3}$ with linear warmup over the first 500 iters
- 8000 iters (vs. 6000)
- Same training data: 20000 examples, batch 32

Same depth, same task, same data volume. Only the width, the head count, the PE, and the LR schedule change. Results:

M	Sinusoidal (baseline)	Learned PE alone	Kitchen-sink recipe
16	0.747	1.000	1.000
24	0.687	0.605	0.998
32	0.664	0.645	0.680

The $M = 24$ row is the headline. The combined recipe lifts test accuracy from 60.5% to 99.75%: a near-complete solve at that scale. The inductive bias was always present in the architecture; the production defaults are what let it become a learned routine.

At $M = 32$ the recipe helps only a little (68.0% versus 64.5% for learned PE alone) and does not solve the task. This is the interesting case, so we chased it down. There are three obvious knobs left to turn: more iterations, more width, more depth. We tried each in isolation.

To run a proper sweep without waiting hours per configuration, we reimplemented the same architecture in PyTorch (`examples/pytorch/pointer_sweep.py` and `pointer_depth.py`). A caution that matters: the PyTorch model uses different initialization and internals from the from-scratch NumPy code, so its absolute accuracies are *not* comparable to the NumPy numbers above. Read the PyTorch results only against each other. The full log of every run, both frameworks, is in `examples/RESULTS.md`.

More iterations alone: no. The NumPy kitchen-sink at $M = 32$ run to 30000 iterations reached only 0.7625. A PyTorch 2-layer model run to 40000 iterations sat at 0.616 with a loss curve essentially flat from iteration 2000 onward. Time is not the bottleneck.

More width alone: no. Doubling to $d_{\text{model}} = 256$ with 8 heads (still 2 layers, $4\times$ the parameters) reached 0.565, indistinguishable from chance-plus. Capacity is not the bottleneck.

More depth: yes. Going from 2 to 3 layers, holding $d_{\text{model}} = 128$, heads = 4, and everything else fixed, produced a clean phase transition and 0.946 test accuracy, still climbing when we stopped it. The controlled comparison (only the layer count varies, same width, heads, data, seed, and 40000-iter budget):

Layers	Parameters	Loss at 40k	Test accuracy	Loss curve
2	270k	0.646	0.616	flat, never transitions
3	403k	0.115	0.946	transitions at iter ~7000

Depth is the knob at $M = 32$, decisively, and neither width nor training time substitutes for it.

It is tempting to package this as a tidy law: “depth equals the number of address bits the model must compose, so larger M needs more layers.” Resist it. The data refute the clean version. $M = 24$ and $M = 32$ have the *same* address width ($\lceil \log_2 24 \rceil = \lceil \log_2 32 \rceil = 5$), yet 2 layers solved $M = 24$ to 0.9975 and cannot solve $M = 32$ at all. The address-decode work is identical across the two; what differs is the **lookup fan-out**, the number of memory cells the second-stage query must discriminate among (24 versus 32), together with the fact that $M = 32$ exercises the full 5-bit address space while $M = 24$ touches only 24 of the 32 codes. The bottleneck at $M = 32$ is the precision of the 1-of- M selection, not the depth of the address decode. What the third layer actually buys is the subject of the next post, which dissects a trained 3-layer model: the short version is that the extra depth goes into a *two-stage* address aggregation (layers 1 and 2 both attend to the address bits) feeding a dereference in layer 3, and that the clean single-head pointer of the $M = 8$ model does not survive the scale-up. The honest takeaway here is the controlled-experiment claim (“at $M = 32$, depth solves where width and time do not”), not a closed-form depth law.

This is the scaling-law picture in miniature: capacity, depth, data, and iterations co-scale, and the binding constraint shifts as the task hardens. At $M = 8$ the binding constraint was nothing (any 2-layer model solves it). At $M = 16$ it was positional-encoding scale. At $M = 24$ it was the combined production recipe. At $M = 32$ it is depth. Each regime hides a different bottleneck, and finding it is the whole game.

11. Inductive bias, three axes (closing)

Across the series we have walked the two axes of supervised inductive bias from the foundations post’s §7:

- **Architecture** (this post and the previous two on CNN, RNN): what structural prior the network commits to about *how features compose*. CNN: locality and translation equivariance. RNN: time-translation equivariance and bottleneck state. Transformer: content-addressable lookup, repeated and composed via depth.
- **Output head** (the link-functions post): what prior the interpretation commits to about *the distribution of the response*.

The transformer-pointers experiment forces us to name a practical third dimension:

- **Implementation realization:** even when the right inductive bias is structurally present in the architecture, the choice of init, positional encoding, supervision shape, and optimizer state can keep it from materializing during training. At scale (and with the standard production defaults) this dimension is mostly invisible. At small experimental scale it becomes the dominant factor, and it is where most of the debugging cycles in real-world transformer engineering go.

Every successful supervised network in modern practice is a triple: a body that bakes in the right architectural prior for the data, a head that bakes in the right likelihood prior for the response, and a training recipe that is co-evolved with the body and head so that the priors can actually be discovered by gradient descent. The combinations multiply.

The transformer is the most flexible body in the series. It bakes in less structure than a CNN or RNN: it does not commit to spatial locality, to temporal recurrence, or to a fixed context window. Instead it commits to a *mechanism*, content-addressable retrieval, and lets the data dictate which positions get retrieved when. The cost of this flexibility is the $T \times T$ attention matrix: quadratic compute and quadratic memory in the sequence length. The benefit is that the same architecture works for any task where the relevant structure is “look at the right past position.” Language modeling becomes the canonical case (where the right past position depends on the syntax, the topic, the entity being discussed), but the same prior is right for code completion, reasoning over context, retrieval-augmented generation, and many other tasks.

But the cost is *also* the implementation-realization burden: the flexibility is genuinely accessible only with a co-evolved training recipe, and the rest of modern transformer practice (warmup, learned or rotary PE, careful init, layer-norm placement, layer-wise lr decay, mixed precision, gradient clipping) is the gradually-accumulated body of tricks that keeps the inductive bias usable. We rediscovered one slice of that recipe by debugging.

The next post takes the model we just trained and reverse-engineers its circuit, asking whether the architectural argument from §3 is empirically vindicated and how the resulting circuit relates to **induction heads** and **in-context learning**. The model is small enough to interpret fully; the answer turns out to clarify what the transformer’s content-addressable-lookup primitive actually is.

The closing post on **reinforcement learning** then introduces the third learning paradigm beyond supervised. There the heads-as-bias frame still applies (a policy head is softmax over actions; a value head is identity + MSE on returns), but the training signal stops being a per-example label and becomes a scalar reward over trajectories. The inductive-bias frame extends naturally: reward shaping, policy architecture, and exploration strategy are all priors. AIXI, the theoretical optimum, is Solomonoff induction (the LM post’s theoretical north star) plus Bayesian decision theory plus reward maximization, all bolted together. The dedicated RL series will unpack each piece.

Appendix: a richer DGP that did not pan out

The first design for this post used a recursive bit-stream DGP with prefix-coded instructions (`examples/bit_dgp.py`): a continuous bit stream where each instruction was either a literal bit (prefix code 0 for value 0 or 10 for value 1) or a deref (11 plus an Elias-gamma-encoded offset plus the deref’s own value codeword), with the offset counted in *instruction* units. Multi-hop chains arose naturally when a deref’s target happened to be another deref.

The DGP is rich and pedagogically appealing: the model has to learn the prefix code, parse the bit stream into instructions, count instructions for the gamma-encoded address, and do the lookup, all from raw bits. Unfortunately, small transformers (`d_model = 64-128`, 1-2 layers) did not learn it in our training budget; even at 5000 iterations the models barely beat the marginal-prediction baseline. The combination of parsing, counting, and addressing in a single small model appears to require more capacity or much longer training than we have available. The PE-scale issue uncovered in section 9 likely contributed; we did not retry the recursive DGP with learned PE.

We kept the generator (`bit_dgp.py`) as reference because the design is interesting in its own right: it is the smallest recursive DGP we know that exercises stacked attention end-to-end without any task-specific scaffolding. Demonstrating that a sufficiently large (or sufficiently well-tuned) transformer can learn it is left as an open exercise.